

Programming styles

BIE-PA2 – Programming and Algorithmic II

Ladislav Vagner

Department of Theoretical Computer Science
Faculty of Information Technology
Czech Technical University in Prague
`xvagner@fit.cvut.cz`

February 23, 2026

Overview

- Programming styles:
 - naive,
 - procedural,
 - object-oriented.
- Classes and objects.
- Difference `class` vs. `struct`.
- Keyword `this`.
- Immutable (`const`) objects and methods.
- Constructors, default constructor.
- Local declarations.
- Static methods and static variables.

Programming styles

- Example – sort an array of numbers.
- Three different implementations:
 - naive style,
 - procedural style,
 - object-oriented style.

Naive style

Naive style guidelines – virtually none:

- do it whatever way,
- use any momentary idea,
- do not waste time with analysis,
- copy-and-paste increases productivity.

Naive style

```
#include <iostream>

int main() {
    size_t size = 0, cap = 0;
    int x, *data = nullptr;

    std::cout << "Enter numbers:" << std::endl;
    while (std::cin >> x) {
        if (size >= cap) {
            cap = cap * 2 + 2;
            int *new_data = new int[cap];
            for (size_t i = 0; i < size; i++)
                new_data[i] = data[i];
            delete[] data;
            data = new_data;
        }

        data[size++] = x;
    }

    . . .
}
```

Naive style

```
...  
if (!std::cin.eof()) {  
    std::cout << "Invalid input" << std::endl;  
    delete[] data;  
    return 1;  
}  
  
for (size_t j = 0; j < size; j++)  
    for (size_t i = 1; i < size; i++)  
        if (data[i - 1] > data[i])  
            std::swap(data[i - 1], data[i]);  
  
std::cout << "Sorted:" << std::endl;  
for (size_t i = 0; i < size; i++)  
    std::cout << data[i] << std::endl;  
  
delete[] data;  
return 0;  
}
```

Naive style – summary

- Program is not structured into functions (mostly).
- Program does not use structured data types (`struct`).
- Operations are programmed "in place".
- Redundant code.

Naive style – summary

- Program is not structured into functions (mostly).
- Program does not use structured data types (`struct`).
- Operations are programmed "in place".
- Redundant code.
- Advantages:
 - short programs are developed quickly,
 - minimal overhead.

Naive style – summary

- Program is not structured into functions (mostly).
- Program does not use structured data types (`struct`).
- Operations are programmed "in place".
- Redundant code.
- Advantages:
 - short programs are developed quickly,
 - minimal overhead.
- Disadvantages:
 - impossible for large programs,
 - programs are difficult to debug, test, maintain, and improve,
 - teamwork is almost impossible.

Procedural style

- Procedural style guidelines:
 - divide the problem into subproblems,
 - use abstraction (abstract command) to describe the subproblem,
 - implement the subproblem in a function/procedure.
- Subproblems in our example:
 - reading the input,
 - adding an element into the array,
 - sorting the array,
 - printing the array.

Procedural style – example

```
#include <iostream>

struct Array {
    size_t size, cap;
    int *data;
};

void a_init(Array& a) { a = {}; }

void a_destroy(Array& a) { delete[] a.data; }

size_t a_size(const Array& a) { return a.size; }

int& a_at(Array& a, size_t i) { return a.data[i]; }
int a_at(const Array& a, size_t i) { return a.data[i]; }

void a_print(const Array& a, std::ostream& out = std::cout) {
    for (size_t i = 0; i < a_size(a); i++)
        out << a_at(a, i) << std::endl;
}

...

```

Procedural style – example

```
...  
void a_push(Array& a, int x) {  
    if (a.size >= a.cap) {  
        a.cap = a.cap * 2 + 2;  
        int *new_data = new int[a.cap];  
        for (size_t i = 0; i < a.size; i++)  
            new_data[i] = a.data[i];  
        delete[] a.data;  
        a.data = new_data;  
    }  
    a.data[a.size++] = x;  
}  
  
void a_sort(Array& a) {  
    for (size_t j = 0; j < a_size(a); j++)  
        for (size_t i = 1; i < a_size(a); i++)  
            if (a_at(a, i - 1) > a_at(a, i))  
                std::swap(a_at(a, i - 1), a_at(a, i));  
}  
...
```

Procedural style – example

```
...  
bool a_read(Array& a, std::istream& inp = std::cin) {  
    int x;  
    while (inp >> x) a_push(a, x);  
    return inp.eof();  
}  
  
int main() {  
    Array numbers;  
    a_init(numbers);  
  
    std::cout << "Enter numbers:" << std::endl;  
    if (a_read(numbers)) {  
        a_sort(numbers);  
        std::cout << "Sorted:" << std::endl;  
        a_print(numbers);  
    } else std::cout << "Invalid input" << std::endl;  
  
    a_destroy(numbers);  
    return 0;  
}
```

Procedural style – summary

- Program is structured into functions/procedures.
- Program may use structured data types (not shown here).
- Redundant code may be eliminated.
- Procedures either use global variables or require all data passed as parameters.

Procedural style – summary

- Program is structured into functions/procedures.
- Program may use structured data types (not shown here).
- Redundant code may be eliminated.
- Procedures either use global variables or require all data passed as parameters.

Advantages:

- redundant code eliminated – easier testing and debugging,
- teamwork: procedures may be simultaneously developed by different programmers.

Procedural style – summary

- Program is structured into functions/procedures.
- Program may use structured data types (not shown here).
- Redundant code may be eliminated.
- Procedures either use global variables or require all data passed as parameters.

Advantages:

- redundant code eliminated – easier testing and debugging,
- teamwork: procedures may be simultaneously developed by different programmers.

Disadvantages:

- the name of the type makes the function names longer,
- more difficult to extend (add new parameters to almost all functions or we need auxiliary `structs`).

Object oriented programming

- The term was first used by Alan Kay:

OOP to me means only messaging, local retention and protection and hiding of state-process, and extreme late-binding of all things. It can be done in Smalltalk and in LISP. There are possibly other systems in which this is possible, but I'm not aware of them.

– Alan Kay

Object oriented programming

- The term was first used by Alan Kay:

OOP to me means only messaging, local retention and protection and hiding of state-process, and extreme late-binding of all things. It can be done in Smalltalk and in LISP. There are possibly other systems in which this is possible, but I'm not aware of them.

– Alan Kay

- Its meaning evolved over time...

Object oriented programming

- The term was first used by Alan Kay:

OOP to me means only messaging, local retention and protection and hiding of state-process, and extreme late-binding of all things. It can be done in Smalltalk and in LISP. There are possibly other systems in which this is possible, but I'm not aware of them.

– Alan Kay

- Its meaning evolved over time...
- Later it was even standardized:

object-oriented

pertaining to a technique or a programming language that supports objects, classes, and inheritance

– ISO/IEC 2382:2015

OOP style

- OOP style guidelines:
 - Analyze the problem and identify participating entities.
 - Analyze interactions between the entities.
 - Implement the entities as classes and objects. The interactions will result in the interfaces (methods) of the classes.
- Object properties:
 - The object holds some information (a state). The state is determined by the content of its data fields (attributes).
 - The interface of an object defines its behavior. The interface is the set of operations (methods) available for the object.
 - The class of an object defines the interface and the data fields of an object. Programs usually create many objects of the same class.
- Compared to the procedural style:
 - Data and functions are bundled into objects.
 - More of the state is implicit.

OOP style – class Array

- Attributes:
 - the number of elements,
 - the number of allocated elements,
 - pointer to allocated space.
- Interface:
 - Construction (and destruction),
 - get the number of elements,
 - get the element at the given index,
 - append an element,
 - sort the array,
 - read the array from input,
 - print the array to output.

- Graphical representation:

Array
size_ cap_ data_
size at push sort read print

OOP style – class Array

```
struct Array {  
    Array(); // Constructor  
    ~Array() { delete[] data_; } // Destructor  
  
    Array(const Array&) = delete; // Disable copying  
    Array& operator = (const Array&) = delete;  
  
    size_t size() const { return size_; }  
  
    int& at(size_t i) { return data_[i]; }  
    int at(size_t i) const { return data_[i]; }  
  
    void push(int x);  
    void sort();  
  
    void print(std::ostream& out = std::cout) const;  
    bool read(std::istream& inp = std::cin);  
  
private:  
    size_t size_, cap_;  
    int *data_;  
};  
.  
.  
.
```

OOP style – class Array

```
...  
Array::Array() { // Suboptimal implementation  
    size_ = cap_ = 0;  
    data_ = nullptr;  
}  
  
void Array::push(int x) {  
    if (size_ >= cap_) {  
        cap_ = cap_ * 2 + 2;  
        int *new_data = new int[cap_];  
        for (size_t i = 0; i < size_; i++)  
            new_data[i] = data_[i];  
        delete[] data_;  
        data_ = new_data;  
    }  
  
    data_[size_++] = x;  
}  
...
```

OOP style – class Array

```
...  
void Array::sort() {  
    for (size_t j = 0; j < size(); j++)  
        for (size_t i = 1; i < size(); i++)  
            if (at(i - 1) > at(i))  
                std::swap(at(i - 1), at(i));  
}  
  
void Array::print(std::ostream& out) const {  
    for (size_t i = 0; i < size(); i++)  
        out << at(i) << std::endl;  
}  
  
bool Array::read(std::istream& inp) {  
    int x;  
    while (inp >> x) push(x);  
    return inp.eof();  
}  
...
```


OOP style – class Array

```
...  
int main() {  
    Array numbers;  
  
    std::cout << "Enter numbers:" << std::endl;  
    if (!numbers.read()) {  
        std::cout << "Invalid input" << std::endl;  
        return 1;  
    }  
  
    numbers.sort();  
    std::cout << "Sorted:" << std::endl;  
    numbers.print();  
  
    return 0;  
}
```

Classes and objects – designer point of view

- Class is an abstraction of a real-world entity.
- Example: class Faculty:
 - all faculties do have some common properties (e.g. a name, an address, ...),
 - all faculties do have some common interface (e.g. they provide study programs, ...).
- Object is one instance of that entity.
- Examples of the Faculty class instances:
 - FIT,
 - FEL,
 - ...

Classes and objects – programmer point of view

- Class is a description of a data type:
 - name of the data type,
 - member variables (fields),
 - interface (methods).
- Classes are developed by programmers and are compiled into the executable program. In C++, new classes cannot be created in run-time.
- The total number of different classes in a typical program is low (hundreds, thousands).
- Objects (class instances) are variables:
 - every object has a class,
 - objects are created and destroyed in run-time,
 - we usually create many objects of the same class,
 - like any other variable, C++ permits statically allocated objects, dynamically allocated objects, arrays of objects, ...

Classes – syntax

- Class declaration:

```
struct Array {  
    Array(); // constructor declaration  
    Array(int x); // with arguments  
    ~Array(); // destructor declaration  
  
    // method (member function) declaration  
    // type name(arguments) const?  
    size_t size() const;  
  
    size_t size_; // member variable declaration  
};
```

- Constructor definition:

```
Array::Array() { ... }
```

- Destructor definition:

```
Array::~~Array() { delete[] data_; }
```

- Method definition:

```
size_t Array::size() const { return size_; }
```

Classes – syntax

- Inline methods:

```
struct Array {  
    size_t size() const { return size_; }  
    Array() { ... }  
    ~Array() { delete[] data_; }  
};
```

- Methods defined in the class are implicitly inline. Methods can also be explicitly marked inline using `inline`.
- Automatically generated and methods explicitly defaulted (= `default`) inside the class definition are also inline.
- More about `inline` and defaulted methods below.
- Note:
 - method declarations are terminated by a semicolon;
 - method definitions are not terminated by a semicolon.

Classes – encapsulation

- Encapsulation:

- `public` – members may be accessed by anyone,
- `protected` – members may be accessed by the class itself and the inherited classes,
- `private` – members may be accessed by the class itself only.

- Example:

```
class T {  
    // members with default access (private here)  
    public:    // members accessible to anyone  
    private:  // members accessible to the class only  
};
```

- Both `class` and `struct` may be used to declare a class.
- The only difference is the default visibility:
 - `private` for `class`,
 - `public` for `struct`.

Objects – accessing methods and member variables

- Class members are accessible using their name inside the class (including non-inline method definitions).

```
size_t Array::size() const { return size_; }
```

- To access members from outside the dot notation must be used (like `struct` in C).
- Methods may be overloaded. The rules for overloaded functions apply (best matching of actual and formal parameters).

```
class T {  
    public:  
    void foo() { ... } // method  
    int bar;           // member variable  
};  
  
T x;  
x.foo();  
x.bar = 10;
```

Objects – `this` keyword

- Local declarations may conflict with member variable names. In such a case, static scoping rules apply.
- Member variables may be accessed using `this` keyword or a fully qualified name.
- It is always a good idea to avoid conflicting names (e.g. `m_` prefix or `_` suffix).

```
struct T {  
    int a;  
    void f(int a);  
};  
  
void T::f(int a) { // shadows T::a  
    T::a = a;      // use qualified name,  
    this->a -= 10;  // this pointer  
    this->T::a += 2; // or even both  
}
```


Constant objects and `const` methods

- Methods, in general, may modify member variables.
- An object can be marked constant – its member variables cannot be modified.
- Methods can be marked `const`:
 - the body of the method cannot modify instance member variables (the compiler checks for that),
 - `const` methods may be invoked on constant objects.
- Non-`const` methods cannot be called for `const` objects.

```
struct Array {  
    size_t size() const; // const method  
    void push(int x);  
    ...  
};  
  
const Array a;           // const object  
std::cout << a.size();   // ok - size is const method  
a.push(5);               // error - a is const but push is not
```

Constant objects and `const` methods

- It is possible (and often needed) to have method overloads which differ only in constness.
- The compiler prefers the non-`const` version.
- The access to elements of a container is the usual example:
 - Non-`const` version returns a reference which allows modification of the stored object.
 - `const` version return by value (or `const` reference) so modification is not possible.

```
struct Array {  
    int& at(size_t i);  
    int at(size_t i) const;  
    ...  
};  
Array a;  
a.push(5);  
a.at(0) += 3; // ok - non-const version returns int&  
std::cout << ((const Array&)a).at(0); // ok - const version
```

Classes – creating objects

- Object is a variable (an instance of a class).
- When an object is created, a constructor is executed.
- Constructors may be overloaded. The rules for overloaded functions apply.
- Constructors cannot be called directly on an existing instance.
- Destructor is automatically called when object is destroyed.
- Examples of statically allocated objects:

```
int foo() {  
    T x(a, b); // Constructor with parameters (a,b) used  
    T y;       // Constructor without parameters  
  
    ...  
}
```

Objects – allocated dynamically

- Object may be allocated dynamically – using the `new` operator. Operator `new` calls the corresponding constructor.
- Dynamically allocated objects are freed using `delete` operator, the operator calls the destructor.
- C allocation (`malloc/free`) cannot be used (constructor/destructor is not called).

```
struct T {  
    int a;  
    T(int x);  
    ~T();  
    void f(int x);  
};  
T *p = new T(20);  
p->a = 10;  
p->f(15);  
delete p;
```

Objects – constructor calls

- Constructor is automatically called when a new object is created.
- When the object is created, further constructor calls are not allowed.
- Explicit call to a constructor creates a new unnamed temporary object. The temporary object may be used for computation (e.g. parameter passing, local copy, ...).
- The temporary object is destroyed at the end of statement.

```
struct T {  
    int p, q;  
    T() { p = 0; q = 0; }  
    T(int fp) { p = fp; q = 0; }  
    ...  
};  
T a, b(1);  
a.T(1);    // error, further constructor calls are not allowed  
a = T(20); // new temporary object is constructed, copied to a,  
           // and then the temporary object is destroyed
```

Objects – constructor calls

- We want an initialized variable of type T:

```
double x = 4;  
T foo(int(x));
```

- **Caution:** This is a declaration of a function, not variable!
- This is called the most vexing parse.
- How to fix it? Use different syntax:

```
T foo1{int(x)};  
T foo2 = {int(x)};  
T foo3 = T{int(x)};  
T foo4 = T(int(x));
```

- All foos behave the same given Foo does not have constructor with a single argument of type `std::initializer_list`.
- If it does, this constructor is strongly preferred in all cases except `foo4`.

Objects – constructor implementation

- All member variables must be initialized when the opening { of the constructor is reached.
- They are initialized with their default constructors unless specified otherwise.
- It does not make sense to initialize a member variable just to overwrite it afterwards. Hence the implementation of `Array::Array` is suboptimal:

```
Array::Array() {  
    size_ = cap_ = 0;  
    data_ = nullptr;  
}
```

- How to invoke a constructor of a member variable?

Objects – constructor implementation

- How to invoke a constructor of a member variable?
- A special syntax is required:

```
struct Array {  
    Array() : size_(0), cap_(0) {}  
    Array(int v) : Array() { push(v); } // delegate  
    size_t size_ = 1, cap_;  
    int *data_ = nullptr;  
};
```

- If present, the initialization after : (so called initializer list) is used instead of the initialization specified in the member declaration.
- Member variables are always initialized in the order of their declaration in the class body.
- Member variables are destroyed after the body of class destructor is completed and they are destroyed in the opposite order than initialized.

Objects – default constructor

- Default constructor is a constructor which may be called without parameters, i.e.:
 - a constructor without formal parameters or
 - a constructor where all formal parameters have default values.
- Default constructor is used if a new object is created and the implementation does not specify parameters.

```
struct T {  
    T(int a = 0) { ... } // a sample default constructor  
    ...  
};  
  
// default constructor is used in all examples below  
T a;  
T b[10]; // default constructor called 10 times  
T *p = new T;
```

Objects – generated methods

- The compiler generates:
 - A default constructor if no constructor is declared. It calls the default constructors of all member variables.
 - A destructor (with empty body).
 - A copy (and move) constructor and assignment operator.
These perform the given operation member-wise. This behaviour is usually wrong for pointer. (More details later.)
- The specifier = `default` forces the compiler to generate the annotated method even if the compiler would not do so otherwise.
- The generation of a method can be suppressed by = `delete`.

```
struct Array {  
    Array() = default;  
    Array(int) { ... }  
    // Disable copying  
    Array(const Array&) = delete;  
    Array& operator = (const Array&) = delete;  
};
```

Local declarations in a class

- A class may host constant and/or data type declarations.
- The declarations follow scoping rules. Moreover, if the declarations are **public**, fully qualified names may be used to access the constants or data types from outside the class.
- Example – we may simplify the modification of Array for different element type:

```
struct Array {  
    using T = int; // T is an alias of int  
    T& at(size_t i);  
    ...  
    T *data_;  
};
```

Keyword `inline`

- Originally the keyword `inline` was a compiler hint to include the code of the function at the call site instead of emitting a call instruction.
- This is a very important optimization. Mainly not because its direct impact but because it enables other optimizations on the inlined code.
- Compilers ignore `inline` as a hint today.
- The compiler must have the code to be able to inline it.
- This has implications:
 - Each compilation unit can include a definition of an inline function. (It must include it for inlining to take place.)
 - All these definitions must be identical.
 - Inline functions are (usually) defined in header files.
 - The compiler (linker) deduplicates the definitions so the final code contains only one copy.

Keyword `inline`

- This has implications:
 - Each compilation unit can include a definition of an inline function. (It must include it for inlining to take place.)
 - All these definitions must be identical.
 - Inline functions are (usually) defined in header files.
 - The compiler (linker) deduplicates the definitions so the final code contains only one copy.
- The main use case of `inline` today is to allow multiple definitions.
- Global and static class variables can be marked `inline` since C++17.
- Such variables may be defined in header files.

Static (class) and instance (member) variables

- Instance (member) variables:
 - declared without any special keyword,
 - every instance has its own set of instance variables.
- Static (class) variables:
 - declared using `static` keyword,
 - static variables are shared among all class instances,
 - static variables must have memory reserved – we either define them outside class declaration or we use `inline`,
 - the access to static variables follows the standard encapsulation rules (`public/protected/private`).

```
class T {  
    int a;                // instance member variable  
    static inline int cnt = 0; // static member variable  
public:  
    T(int x) { a = x; cnt++; }  
    ~T() { cnt--; }  
};
```

Class (static) and instance methods

- Instance methods:
 - can only be invoked on an existing object,
 - has the access to member variables in the instance directly or via `this` pointer,
 - the method may access class variables,
 - the method may call other instance/class methods.
- Class (static) methods:
 - are similar to the ordinary functions,
 - the method does not operate on any implicit instance, thus neither implicit member variables nor `this` pointer is available.
 - the method may access class variables and may call other class methods.

```
class T {  
    static inline int cnt = 42;  
public:  
    static int getCount() { return cnt; }  
};  
cout << T::getCount(); // static method invocation
```

What we did not cover

- inheritance,
- `friend`,
- lvalue (&) and rvalue (&&) methods,
- `mutable`,
- pointer-to-member.

Questions ...